

PRJ10 – Sistema para Gerenciamento de Recursos Humanos em Pesquisa

Relatório Final: Marco 2 (OCRA – Operações, Consultas, Rotinas e Administração)

**Daniel Antônio de Matos Fernandes¹, Marcelo Moura de Carvalho Santana¹,
Paulo Roberto de Matos Trigo¹, Victor Manoel Conceição de Almeida¹**

¹ Instituto de Computação – Universidade Federal da Bahia (UFBA)
Salvador – BA – Brasil

`{danielmatos, marcelomcs, victor.manoel, paulormt}@ufba.br`

Resumo. *Este relatório descreve o Marco 2 do projeto PRJ10, desenvolvido em PostgreSQL para gerenciamento de recursos humanos, contratos, bolsas, financiadores e produção acadêmica em projetos de pesquisa. O documento consolida as correções realizadas após o Marco 1, a organização final dos artefatos SQL, as rotinas transacionais, as dez consultas de apoio a dashboards, duas visões materializadas, duas stored procedures, duas transações explícitas, uma trigger de validação de regra de negócio, políticas de acesso e estratégia de backup. O foco da entrega é a camada de banco de dados, sem dependência de scripts externos de manipulação.*

1. Descritivo do Projeto e Justificativas

O PRJ10 modela um sistema de apoio à gestão de projetos de pesquisa. O banco permite registrar projetos, financiadores, pesquisadores, bolsas, contratos, publicações, relatórios e valores aportados. A motivação do sistema é dar rastreabilidade ao ciclo completo de um projeto acadêmico: entrada de recursos, formalização de vínculos, alocação de bolsas e produção científica resultante. O Marco 2 partiu do modelo produzido no Marco 1, mas incorporou correções estruturais e operacionais solicitadas para a continuidade do projeto.

1.1. Revisão e Justificativa do Minimundo

No primeiro marco, estendemos os requisitos básicos propostos pelo estudo de caso original para abarcar um cenário aderente à realidade das universidades públicas. A justificativa para essa extensão do minimundo reside na necessidade de rastreabilidade de fomento. No escopo inicial, não havia distinção refinada de como o dinheiro ingressava e era gasto.

Para suprir isso, justificamos a criação de entidades separadas como `bolsa` e `financiador`. A abstração da `bolsa` como um fundo associado ao projeto (e não diretamente ao pesquisador) permite que o sistema gerencie bolsas ociosas, histórico de substituição de bolsistas (através da tabela de `contratos`) e análises segmentadas de investimento público versus privado. Além disso, incluímos `relatorios` periódicos, pois os artefatos de governança exigem prestação de contas vinculada à continuidade do financiamento.

1.2. Crítica ao Modelo e Resolução do Loop

Avaliando criticamente a modelagem entregue na fase anterior, admitimos a existência de um *loop estrutural* entre as entidades projeto, contrato e bolsa, mas não se trata de um *loop lógico*. O contrato pertence a um projeto e referencia uma bolsa; contudo, a bolsa também possui seu próprio vínculo com o projeto.

Em vez de unificar as entidades, que prejudicaria a semântica de negócio, já que bolsas podem existir sem contratos ativos e pesquisadores podem atuar como voluntários, optamos por manter as abstrações isoladas e tratar o risco de anomalia na camada de dados. Para mitigar o loop, garantindo que um pesquisador não assuma via contrato uma bolsa pertencente a um projeto distinto, implementamos a *Trigger* mitigadora `tg_valida_bolsa_contrato`, blindando a integridade do banco.

1.3. Requisitos Funcionais do Sistema

A base do sistema é delimitada por seis requisitos originais e três requisitos estendidos propostos para garantir a completude do domínio.

Requisitos Originais:

- **(R1)** Cadastrar e gerenciar os órgãos financiadores externos dos projetos de pesquisa.
- **(R2)** Controlar as bolsas de pesquisa vinculadas e custeadas por cada projeto.
- **(R3)** Formalizar o repasse de bolsas a pesquisadores por meio de contratos com vigência determinada.
- **(R4)** Manter o cadastro de pesquisadores alocados nos projetos.
- **(R5)** Registrar a produtividade científica gerada pelos projetos (publicações).
- **(R6)** Emitir relatórios periódicos de acompanhamento e progresso científico.

Em resposta às avaliações e críticas construtivas recebidas na etapa anterior (Marco 1), os requisitos estendidos foram revisados e aprofundados. Essa adequação tem o objetivo de garantir maior integridade relacional, rastreabilidade e aderência à complexidade administrativa e fiscal do ambiente de pesquisa.

Requisitos Estendidos (Revisados e Justificados):

- **(R7) Histórico Contratual e Retenção de Vínculos:** Assegurar a preservação de todo o histórico de vínculos entre pesquisadores e projetos por meio de uma entidade independente. Essa abordagem previne a perda de dados temporais por exclusões indevidas ou sobreposições, garantindo a rastreabilidade completa da trajetória acadêmica e profissional dos pesquisadores ao longo do tempo.
- **(R8) Controle Financeiro e Monitoramento Orçamentário:** Viabilizar o acompanhamento rigoroso e a análise do custo efetivo das bolsas em relação ao montante total de financiamento aportado. Este requisito é vital para assegurar a saúde financeira dos projetos, facilitando o cruzamento de dados de custeio e impedindo alocações que ultrapassem os limites do fomento recebido.
- **(R9) Segmentação Jurídica e Fiscal do Financiamento:** Estratificar os dados de fomento com base na natureza jurídica do órgão financiador, diferenciando de forma estrita os recursos de capital público e privado. Essa tipificação viabiliza auditorias precisas, adequação a marcos regulatórios distintos e a emissão de relatórios de prestação de contas altamente segmentados.

1.4. Correções Incorporadas do Marco 1

As principais correções em relação à primeira versão foram:

- adoção consistente de nomes em *snake_case*, evitando nomes herdados de mapeamentos automáticos, como colunas com prefixo `fk_`;
- uso de chaves substitutas com `SERIAL` para entidades fortes, como `projeto`, `pesquisador`, `financiador`, `bolsa`, `contrato` e `publicacao`;
- correção da relação entre `bolsa` e `pesquisador`: a `bolsa` pertence ao `projeto`; a associação com o `pesquisador` ocorre pelo `contrato`;
- carga SQL nativa em `03_DML_Carga_Dados.sql`, usando as funções `WITH RECURSIVE`, `RANDOM()` e instrução `INSERT INTO ... SELECT`, substituindo a carga externa;
- separação dos artefatos antigos do Marco 1 na pasta `Arquivos_Marco_1`, mantida apenas como referência histórica e não como parte do fluxo executável do Marco 2.
- **Expansão Metodológica do Plano de Indexação:** Em resposta direta à penalidade aplicada no Marco 1 por escassez de cenários experimentais (apenas um plano básico fornecido), a equipe revisitou a etapa de otimização física. Foram concebidos e implementados de forma cumulativa o **Plano 2 (Índices Compostos)** e o **Plano 3 (Índices Parciais)**, cujos impactos na redução de custo algorítmico e ganho real de desempenho computacional encontram-se detalhadamente dissecados e comparados na Seção 7.

A pasta `Arquivos_Marco_1` contém consultas e medições anteriores que servem como histórico do desenvolvimento, mas não substituem os arquivos numerados do Marco 2.

2. Organização da Entrega e Modelo Relacional

A entrega foi empacotada com numeração explícita, facilitando a leitura e a execução em cadeia. A Tabela 1 resume o papel de cada arquivo na arquitetura da solução.

O modelo relacional possui oito tabelas principais: `projeto`, `financiador`, `pesquisador`, `bolsa`, `contrato`, `publicacao`, `relatorio` e `financia`. O centro operacional é a tabela `contrato`, pois ela conecta `pesquisador`, `projeto` e, opcionalmente, `bolsa`. Essa decisão preserva o histórico de vínculos, permite registrar voluntários e bolsistas no mesmo mecanismo e evita que a `bolsa` carregue diretamente a responsabilidade de identificar o `pesquisador`.

3. Carga de Dados e Correção da Dependência Externa

Neste Marco 2, exige-se que a operação do banco seja feita unicamente via SQL. Desse modo, implementamos a carga de dados diretamente no arquivo `03_DML_Carga_Dados.sql`, sem utilizar geradores externos como Python ou Faker. A solução utiliza expressões recursivas para gerar sequências e comandos `INSERT INTO ... SELECT` para popular as entidades de forma autossuficiente. A carga atual cria aproximadamente 1000 projetos, 50 financiadores, 5000 pesquisadores, 3000 bolsas iniciais, 6000 contratos, 1500 registros de financiamento, 2000 publicações e 2000 relatórios.

Tabela 1. Ordem de execução padronizada dos artefatos SQL do Marco 2

Arquivo	Função
01_Tabelas_DDL.sql	Criação das tabelas, chaves primárias, chaves estrangeiras e restrições relacionais.
02_Trigger_Regra_Negocio.sql	Criação da função e da trigger mitigadoras do loop estrutural.
03_DML_Carga_Dados.sql	Carga maciça de dados sintéticos gerada unicamente via SQL nativo.
04_Views_Consultas_Dashboards.sql	Criação das 2 materialized views, 8 consultas em tempo real e comandos de REFRESH.
05_Procedures_Telas_Transacoes.sql	Criação das 2 stored procedures e das 2 transações explícitas.
06_Indices_Basicos.sql	Índices de performance fundamentais para chaves estrangeiras e filtros funcionais.
07_Indices_Compostos.sql	Índices multicoluna para padrões de consulta recorrentes.
08_Indices_Parciais.sql	Índices parciais para cenários específicos e subconjuntos de dados.
09_Politicas_Acesso.sql	Criação de roles, privilégios DCL e manual técnico de backup lógico.

Os contratos vinculados a bolsas recuperam o `id_projeto` da própria bolsa, evitando inconsistência entre `contrato.id_projeto` e `bolsa.id_projeto` durante a geração de dados. Essa refatoração corrige deslize da primeira versão. Na versão atual, a reprodução da base depende apenas do PostgreSQL.

4. Mitigação de Loop Estrutural por Trigger

Conforme introduzido na seção de justificativas, a modelagem possui uma situação que exige controle adicional: `bolsa` pertence a `projeto`, enquanto `contrato` também pertence a `projeto` e pode referenciar uma `bolsa`. Sem uma regra procedural, seria possível inserir um contrato do projeto A usando uma bolsa do projeto B.

Essa mitigação responde diretamente a uma vulnerabilidade de negócio apontada na avaliação do Marco 1. Como a relação lógica e contábil dita que "bolsa e contrato não é a mesma coisa que bolsa e projeto", a *Trigger* (`tg_valida_bolsa_contrato`) atua interceptando a transação em nível relacional para garantir que a abstração lógica do sistema não permita inconsistências físicas.

A trigger é disparada antes de qualquer evento de `INSERT` ou `UPDATE` na tabela `contrato`. Quando `NEW.id_bolsa` não é nulo, a função compara a integridade dos dados e aciona a cláusula condicional restritiva:

```

1 IF NEW.id_projeto <> v_projeto_bolsa THEN
2     RAISE EXCEPTION 'Operacao negada pela regra de negocio';
3 END IF;
```

Listing 1. Trecho central condicional da trigger de validação

Essa solução mantém a separação conceitual entre bolsa, contrato e projeto, mas impede associações fisicamente incoerentes no banco.

5. Consultas Analíticas e Dashboards

O Marco 2 exige dez consultas atuando como *back-end* de dashboards: quatro intermediárias e seis avançadas. A decisão de arquitetura foi aplicar a tecnologia de *Materialized Views* de forma cirúrgica em apenas duas rotinas: a consulta de maior custo computacional do painel estratégico (Dashboard 1) e a consulta de maior complexidade estrutural do painel operacional (Dashboard 2). Essa abordagem evita *overengineering* e custos desnecessários de armazenamento/atualização, preservando a entrega de dados em tempo real nas outras oito consultas operacionais que exigem informações atualizadas no momento do acesso. Assim, o total analítico permanece igual a dez artefatos de consulta no arquivo `04_Views_Consultas_Dashboards.sql`.

Tabela 2. Mapeamento das consultas de dashboard

Painel Analítico	ID	Tipo	Execução	Finalidade
Dashboard 1 (Estratégico)	Q.2.22	Avançada	Tempo real	Financiadores cujos projetos possuem publicações associadas.
	Q.2.24	Avançada	Mat. View	Consolidação anual de aportes públicos e privados.
	Q.2.25	Avançada	Tempo real	Projetos cujo custo de bolsas ultrapassa 60% do aporte.
	Q.2.26	Avançada	Tempo real	Intervalos curtos entre vínculos sucessivos de pesquisadores.
Dashboard 2 (Operacional)	Q.1.12	Intermediária	Tempo real	Contratos por projeto e média de dias de contrato.
	Q.1.13	Intermediária	Tempo real	Bolsas pagas por financiador e total investido em bolsas.
	Q.1.14	Intermediária	Tempo real	Distribuição de pesquisadores e contratos por titulação.
	Q.1.15	Intermediária	Tempo real	Produtividade por projeto, combinando publicações e pesquisadores.
	Q.2.21	Avançada	Tempo real	Pesquisadores com contratos acima da média por titulação.
	Q.2.23	Avançada	Mat. View	Ranking interno de bolsas do projeto utilizando <i>window functions</i> .

As consultas Q.2.23 e Q.2.24 foram escolhidas para materialização por combinarem agregações, pivoteamento, funções de janela e junções complexas. A materialized

view `mv_estrategica_aportes_anuais` consolida aportes por ano e tipo de financiador. A view `mv_operacional_equidade_incentivos` calcula ranking interno de bolsas por projeto. Os comandos `REFRESH MATERIALIZED VIEW` aparecem no arquivo de dashboards e também são chamados pelas procedures, de modo que alterações operacionais atualizem o cache analítico.

6. Rotinas Avançadas e Transações

O arquivo `05_Procedures_Telas_Transacoes.sql` contém exatamente duas stored procedures e duas transações explícitas, sinalizadas com comentários. As procedures representam ações de telas operacionais e obedecem às contagens de DML exigidas.

6.1. Stored Procedure da Tela 1

A procedure `sp_promover_voluntario` representa a promoção de um pesquisador voluntário para bolsista. Ela recebe pesquisador, projeto, valor da bolsa, categoria, número de processo e nova titulação. A rotina executa: 1 `SELECT`, 2 `UPDATE`s e 2 `INSERT`s. Primeiro, localiza contrato voluntário ativo. Em seguida, encerra o contrato antigo, atualiza a titulação do pesquisador, cria a nova bolsa e cria o novo contrato de bolsista. Ao final, atualiza a materialized view estratégica.

6.2. Stored Procedure da Tela 2

A procedure `sp_encerramento_emergencial` representa o encerramento administrativo de um projeto. Ela recebe o projeto e o motivo do encerramento. A rotina executa: 2 `SELECT`s, 1 `DELETE`, 2 `UPDATE`s e 1 `INSERT`. O primeiro `SELECT` contabiliza contratos ativos; o segundo calcula o próximo sequencial de relatório. Em seguida, remove relatórios inválidos, marca o projeto como encerrado, cancela contratos ativos e registra um relatório de encerramento.

6.3. Transações Explícitas

As chamadas das telas são apresentadas em blocos transacionais explícitos. A Tela 1 chama `sp_promover_voluntario` entre `BEGIN` e `COMMIT`. A Tela 2 chama `sp_encerramento_emergencial` também entre `BEGIN` e `COMMIT`. Essa organização evidencia a fronteira ACID no ponto de uso, mantendo as procedures sem controle transacional interno explícito.

Nota sobre a validação transacional: Os comandos de chamada (`CALL`) exemplificados nos scripts da Camada Operacional utilizam identificadores estáticos (e.g., `ID 1` ou `ID 10`) puramente para fins de demonstração da fronteira transacional. Uma vez que a massa de dados do sistema (DML) é gerada de forma sintética e aleatória a cada execução, os registros associados a estes IDs fixos podem não cumprir as premissas da regra de negócio num determinado momento. Portanto, para testar as `Stored Procedures` manualmente e sem erros de integridade, é necessário consultar previamente as tabelas respectivas e substituir os parâmetros por identificadores válidos gerados na sua instância.

7. Plano de Indexação

A arquitetura de otimização foi dividida em quatro fases de medição. Adicionamos ao Marco 2 outros dois planos que faltaram no Marco 1:

- **Baseline:** O estado original da base de dados, sem qualquer índice aplicado (exceto as chaves primárias automáticas).
- **Plano 1 (Básico):** O script `06_Indices_Basicos.sql`, que não é o baseline, mas sim a primeira intervenção de otimização, focada em indexar Chaves Estrangeiras (FKs) que o PostgreSQL não indexa por padrão, além de filtros funcionais simples.
- **Plano 2 (Composto):** O script `07_Indices_Compostos.sql`, focado em otimizar consultas analíticas através de índices multicoluna.
- **Plano 3 (Parcial):** O script `08_Indices_Parciais.sql`, focado em reduzir o custo de armazenamento indexando apenas subconjuntos específicos (ex: bolsistas e aportes de alto valor).

7.1. Índices Básicos e Funcionais

Os índices do Plano 1 cobrem colunas de junção e filtros recorrentes:

- `contrato.id_pesquisador` e `contrato.id_projeto`
- `bolsa.id_projeto` e `publicacao.id_projeto`
- `financia.id_projeto` e `contrato.data_vencimento`

Também foram criados índices funcionais sobre `LOWER(tipo_financiador)` e `LOWER(categoria_bolsa)`, pois várias consultas segmentam dados por tipo de financiador ou categoria de bolsa de forma *case-insensitive*.

7.2. Índices Compostos

A segunda abordagem adicionou índices multicoluna para consultas que filtram ou ordenam por combinações frequentes:

- `idx_comp_contrato_pesq_proj` em `contrato(id_pesquisador, id_projeto)`;
- `idx_comp_bolsa_proj_valor` em `bolsa(id_projeto, valor_bolsa)`;
- `idx_comp_pub_proj_data` em `publicacao(id_projeto, data_publicacao)`;
- `idx_comp_financia_proj_valor` em `financia(id_projeto, valor_aportado)`.

Esses índices atendem consultas de ranking, histórico, análise de financiamento por projeto e produtividade temporal.

7.3. Índices Parciais

Os índices parciais reduzem custo de armazenamento ao indexar apenas subconjuntos relevantes. O arquivo `08_Indices_Parciais.sql` cria índice para contratos de bolsistas, financiamentos acima de R\$100.000,00 e financiadores públicos. Esses filtros aparecem em análises financeiras e operacionais e não exigem indexar todas as linhas da tabela.

7.4. Avaliação de Desempenho (Speedup)

Tabela 3. Avaliação de Desempenho e Speedup (em milissegundos)

Query	Baseline		Plano 2 (Composto)			Plano 3 (Parcial)		
	Média	DP	Média	DP	Speedup	Média	DP	Speedup
Q1.1	25.996	1.447	25.015	1.653	1.039x	27.435	2.902	0.948x
Q1.2	5.694	0.835	5.738	0.516	0.992x	5.366	0.839	1.061x
Q1.3	11.640	2.487	9.276	1.491	1.255x	10.046	0.904	1.159x
Q1.4	6.843	0.540	7.693	1.002	0.889x	7.394	0.799	0.925x
Q1.5	7.232	0.933	9.559	1.432	0.756x	9.048	0.425	0.799x
Q1.6	7.712	1.174	8.512	0.828	0.906x	8.862	0.630	0.870x
Q1.7	3.832	0.729	3.065	0.279	1.250x	3.431	0.471	1.117x
Q1.8	6.816	0.893	6.049	0.366	1.127x	6.138	0.567	1.111x
Q1.9	26.406	3.563	23.805	1.907	1.109x	24.232	2.186	1.090x
Q1.10	3.294	0.482	3.397	0.490	0.970x	3.173	0.430	1.038x
Q1.11	86.465	5.580	88.222	10.140	0.980x	76.993	4.366	1.123x
Q1.12	3.508	0.600	3.762	0.460	0.932x	5.773	0.710	0.608x
Q1.13	2.437	0.462	2.389	0.212	1.020x	2.527	0.398	0.964x
Q1.14	7.042	0.536	7.666	0.665	0.919x	8.220	0.411	0.857x
Q1.15	10.124	1.978	9.503	0.646	1.065x	9.314	0.559	1.087x
Q2.1	8.393	0.969	7.969	0.420	1.053x	7.966	0.638	1.054x
Q2.2	7.584	0.723	5.182	0.638	1.463x	7.247	0.483	1.046x
Q2.3	5.712	0.846	1.767	0.187	3.232x	5.445	0.673	1.049x
Q2.4	11.502	1.280	10.604	0.869	1.085x	10.245	0.480	1.123x
Q2.5	1.480	0.444	1.213	0.202	1.220x	1.263	0.287	1.171x
Q2.6	6.390	0.749	5.634	0.473	1.134x	5.423	0.375	1.178x
Q2.7	1.817	0.306	4.697	0.491	0.387x	1.598	0.296	1.137x
Q2.8	5.352	0.475	2.571	0.216	2.082x	4.833	0.859	1.107x
Q2.9	2.962	0.487	3.622	0.703	0.818x	2.610	0.288	1.135x
Q2.10	3.259	0.381	7.610	0.814	0.428x	3.034	0.321	1.074x
Q2.11	9.052	0.858	5.914	0.762	1.531x	7.693	0.428	1.177x
Q2.12	6.181	0.898	2.736	0.356	2.259x	5.431	0.679	1.138x
Q2.13	2.858	0.325	3.699	0.267	0.773x	2.562	0.418	1.115x
Q2.14	4.027	0.625	3.109	0.262	1.295x	3.870	0.440	1.041x
Q2.15	3.517	0.448	4.358	0.778	0.807x	3.274	0.474	1.074x
Q2.16	4.556	0.765	4.600	0.529	0.990x	4.424	0.738	1.030x
Q2.17	4.662	0.675	1.563	0.249	2.983x	4.229	0.557	1.103x
Q2.18	1.472	0.213	2.560	0.447	0.575x	1.566	0.319	0.940x
Q2.19	2.459	0.239	2.948	0.528	0.834x	2.412	0.199	1.020x
Q2.20	2.954	0.644	4.358	0.778	0.678x	2.560	0.185	1.154x
Q2.21	5997.467	116.768	5868.205	87.750	1.022x	5768.999	89.227	1.040x
Q2.22	1.099	0.113	0.849	0.082	1.295x	0.903	0.123	1.217x
Q2.23	2.022	0.293	1.967	0.153	1.028x	1.828	0.167	1.106x
Q2.24	380.655	30.830	294.711	14.283	1.292x	366.315	27.704	1.039x
Q2.25	7.780	0.616	7.642	0.298	1.018x	7.512	0.551	1.036x

Para validar a eficiência dos novos planos de indexação arquitetados no banco, as consultas do Marco 1, localizadas no arquivo `Queries_Marco_1.sql`, foram executadas 20 vezes para capturar a Média e o Desvio Padrão. O cálculo de *Speedup* demonstrará o ganho ou perda de desempenho partindo do *Baseline* (sem índices) e passando cumulativamente pelos Planos 2 e 3, pois no relatório do Marco 1 já temos a tabela referente ao Plano 1. Os resultados comparativos de todas as execuções, atestando as melhorias de tempo de resposta, encontram-se consolidados na Tabela 3.

8. Políticas de Acesso, Privacidade e Backup

O arquivo `09_Políticas_Acesso.sql` implementa comandos DCL e documenta a estratégia de backup lógico. Foram definidos três perfis: `admin_pesquisa`, `gestor_`

rh e auditor_financeiro.

O perfil `admin_pesquisa` é administrativo. O perfil `gestor_rh` recebe permissões de `SELECT`, `INSERT` e `UPDATE` sobre `pesquisador` e `contrato`, além de leitura em `projeto` e `bolsa` para validações. O `DELETE` sobre `contrato` e `pesquisador` é revogado explicitamente, preservando histórico. O perfil `auditor_financeiro` recebe leitura sobre `financiador`, `bolsa`, `financia` e sobre as `materialized views`, sem acesso direto aos dados pessoais completos dos pesquisadores.

A política de preservação utiliza backup lógico com a ferramenta oficial `pg_dump` em formato customizado (`-F c`), permitindo restauração flexível e parcial. Para garantir a segurança orgânica do SGBD e evitar a vulnerabilidade de exposição de credenciais (*Hardcoded Credentials*), a automação prevista via `crontab` em horário de baixa utilização (madrugada) consome a variável de ambiente do sistema operacional (`PGPASSWORD`).

Em caso de contingência e desastres, o plano de recuperação da base de dados especifica o uso do utilitário `pg_restore` com a flag `-1` (Single Transaction). Essa prática encapsula todo o processo de restauração em uma única transação ACID, assegurando que falhas interruptivas (como queda de conexão ou espaço em disco) acionem um *rollback* completo, impedindo que o banco de dados permaneça em um estado inconsistente ou parcialmente restaurado.

```
1 pg_dump -U admin_pesquisa -h localhost -F c \  
2   -f /var/backups/postgres/prj10_db_$(date +%Y-%m-%d).dump  
   prj10_db
```

Listing 2. Estratégia de backup lógico documentada

9. Rastreabilidade do Barema

A Tabela 4 consolida os itens exigidos pelo Marco 2 e os respectivos arquivos de implementação.

Tabela 4. Rastreabilidade dos itens obrigatórios do Marco 2

Item	Qtd.	Localização
Materialized Views	2	04_Views_Consultas_Dashboards.sql: mv_estrategica_aportes_anuais e mv_operacional_ equidade_incentivos.
Stored Procedures	2	05_Procedures_Telas_ Transacoes.sql: sp_promover_ voluntario e sp_encerramento_ emergencial.
Transações	2	Blocos BEGIN/COMMIT no arquivo de procedures.
Consultas de dashboard	10	04_Views_Consultas_ Dashboards.sql (2 materialized views + 8 consultas em tempo real).
DCL e perfis de acesso	3 roles	09_Politicas_Acesso.sql.

Tabela 4. Rastreabilidade dos itens obrigatórios do Marco 2

Item	Qtd.	Localização
Backup	1 política	pg_dump documentado no arquivo de políticas.

10. Conclusão

O Marco 2 consolidou o PRJ10 não apenas como um repositório estruturado, mas como uma plataforma centrada no SGBD que aplica conceitos avançados de engenharia de dados. Uma análise crítica da arquitetura desenvolvida revela que o encapsulamento das regras de negócio diretamente no banco mitigou drasticamente falhas de integridade. O uso rigoroso de *Stored Procedures* combinadas com o controle transacional explícito (ACID) tornou as operações críticas de RH (como admissão e demissão emergencial de bolsistas) blindadas e à prova de falhas na camada de aplicação.

No eixo de governança e segurança, as políticas de DCL estabelecidas protegeram os dados sensíveis dos pesquisadores, adequando o sistema às premissas da LGPD ao evitar a deleção indevida e ao segmentar as permissões de acesso por perfil (gestores, auditores e administradores).

Ademais, a implementação dos três níveis de indexação comparativos (básico, composto e parcial) endereçou e preparou o terreno para sanar os déficits de performance em grandes volumes analíticos apontados durante a avaliação do Marco 1. A criação estratégica de *Materialized Views* exclusivamente para os dashboards mais dispendiosos equilibrou perfeitamente o *trade-off* entre custo de armazenamento e latência de consulta. Em suma, as rotinas operam de forma autossuficiente e integrada, atestando o domínio prático da equipe sobre os pilares da engenharia avançada de SGBDs relacionais.

Anexos: Repositório e Tutorial de Reprodução

A) Repositório e Códigos-Fonte

Todos os artefatos SQL desenvolvidos ao longo do projeto estão devidamente versionados e encontram-se públicos no repositório oficial no GitHub para o marco 2. Acesse em: <https://github.com/DanHuon/MATA60-PRJ10>

B) Tutorial de Reprodução

Para reproduzir o ambiente em PostgreSQL e garantir a correta interdependência dos objetos do banco de dados, é necessário criar um banco vazio e executar os arquivos estritamente na ordem numerada abaixo. A pasta *Arquivos_Marco_1* deve ser entendida apenas como material histórico, não como fonte de execução do Marco 2.

1. Criar o banco de dados PostgreSQL vazio (ex: prj10_db).
2. Executar *01_Tabelas_DDL.sql* (Criação do esquema físico).
3. Executar *02_Trigger_Regra_Negocio.sql* (Mitigação do loop estrutural).
4. Executar *03_Views_Consultas_Dashboards.sql* (Criação das *Materialized Views* e consultas analíticas).

5. Executar 04_Procedures_Telas_Transacoes.sql (Implantação das rotinas DML e transações).
6. Executar 05_DML_Carga_Dados.sql (População massiva automatizada).
7. Executar os planos de indexação: 06_Indices_Basicos.sql, 07_Indices_Compostos.sql e 08_Indices_Parciais.sql.
8. Executar 09_Politicas_Acesso.sql (Definição de perfis, restrições DCL e documentação de backup).

Nota: O arquivo queries_novas_etapa_2.sql é apenas para visualização isolada das novas consultas analíticas exigidas, não compondo a esteira de dependências e execução obrigatória para a construção do banco.